

EduCore: Especificación de Requisitos del Sistema

Programación III · ISI-BSI-09 · Proyecto 1 — Enunciado

Universidad Americana

Julio 2026

Entrega: 5 de Julio, 2026 · **Tag de entrega:** v1.0-p1 **Modalidad:** grupal · **Valor:** 15 %

0. Contexto: La situación

Pónganse en situación. Lean esta sección antes de saltar a los requisitos. El proyecto no es un ejercicio aislado: es la continuación de un trabajo real que ya estaba en marcha.

La startup educativa **EduCore** está construyendo un sistema para la gestión integral de centros educativos: estudiantes, personal docente y administrativo, edificios, aulas y la oferta académica. El objetivo es reemplazar las planillas de Excel y los cuadernos que las instituciones todavía usan para administrar su día a día.

El **equipo de desarrollo original** arrancó el proyecto y dejó funcionando el primer módulo de extremo a extremo: el **flujo completo de Estudiantes** (Model → View → Controller → DAO), con su arquitectura, sus convenciones y sus pruebas. Antes de poder continuar con el resto del sistema, sin embargo, ese equipo fue **reasignado de forma urgente a otro proyecto de la empresa**, y el desarrollo quedó congelado a mitad de camino.

Ahí entran **ustedes**. Han sido seleccionados como el **nuevo equipo** que tomará el relevo. No empiezan desde cero: heredan un repositorio con código vivo, una arquitectura ya decidida y un módulo de referencia que funciona. Su misión es **continuar el trabajo respetando el estilo y las decisiones del equipo anterior**, entregando los módulos que faltan con la misma calidad y coherencia.

Su ventaja: el módulo de **Estudiantes** ya implementado es su mejor documentación. Estúdienlo, entiendan cómo se conectan sus capas y repliquen ese patrón en los módulos nuevos. Un buen equipo que hereda código no reinventa: aprende del existente y mantiene la consistencia.

A continuación, el enunciado formal del Proyecto 1: lo que el sistema debe cumplir cuando ustedes terminen su parte.

1. Introducción

1.1 Propósito

Este documento especifica los requisitos que el sistema EduCore debe cumplir al término del Proyecto 1. Sirve como referencia única para el diseño, la implementación y la evaluación del trabajo.

1.2 Alcance

El Proyecto 1 extiende el sistema EduCore con dos módulos nuevos: **Empleados** y **Académico**. Cada módulo entrega su capa completa: Model, View, Controller y DAO. El módulo de Estudiantes, ya implementado en el repositorio, actúa como patrón de referencia.

1.3 Glosario

Término	Definición
Model	Clases que representan entidades del dominio. No tienen referencia a la vista ni a la base de datos.
View	Clases que interactúan con el usuario por consola. Extienden VistaBase. No contienen lógica de negocio.
Controller	Coordina la vista con el repositorio. Valida datos, construye objetos y delega la persistencia al DAO.
DAO	Implementa Repositorio<T>. En P1 usa una lista en memoria; en P2 se reemplaza por MySQL sin tocar el Controller.

2. Base del sistema

El repositorio ya contiene los componentes siguientes. No los modifiquen.

2.1 Capa Model

Clase	Descripción
Persona	Clase abstracta raíz. Atributos: id (int), nombre, apellidos, email. Métodos abstractos: getInfo(), getTipo().
Estudiante	Abstracta, extiende Persona. Atributos: carnet, TARIFA_BASE (€150 000). Métodos abstractos: calcularMatricula(), puedeMatricular().
EstudianteRegular	Concreta. calcularMatricula() retorna TARIFA_BASE. puedeMatricular() retorna true.
EstudianteBecado	Concreta. Atributo extra: porcentajeBeca. calcularMatricula() retorna $TARIFA_BASE \times (1 - porcentajeBeca)$.

2.2 Capa DAO

Clase	Descripción
Repositorio<T>	Clase abstracta genérica. Métodos: guardar(T), actualizar(T), eliminar(int id), buscarPorId(int id) → Optional<T>, buscarTodos() → List<T>. Todos declaran throws Exception.
ListaEstudianteRepo	Implementa Repositorio<Estudiante> con ArrayList.

2.3 Capa Controller

Clase	Descripción
EstudianteController	Recibe Repositorio<Estudiante> por constructor. CRUD completo: registrarRegular, registrarBecado, listar, buscarPorId, actualizar, eliminar. Lanza IllegalArgumentException ante datos inválidos.

2.4 Capa View

Clase	Descripción
VistaBase	Abstracta. Utilidades de consola: leerTexto, leerEntero, leerDecimal, leerFecha, mostrarMensaje, mostrarError.
EstudianteView	CRUD completo de estudiantes. Extiende VistaBase. Recibe Repositorio<Estudiante> por constructor.
MenuPrincipalView	Menú principal. Crea los repositorios compartidos y los pasa a cada vista. Opción 1 (Estudiantes) conectada. Opciones 2 y 3 tienen comentarios TODO que el grupo activa al implementar sus módulos.

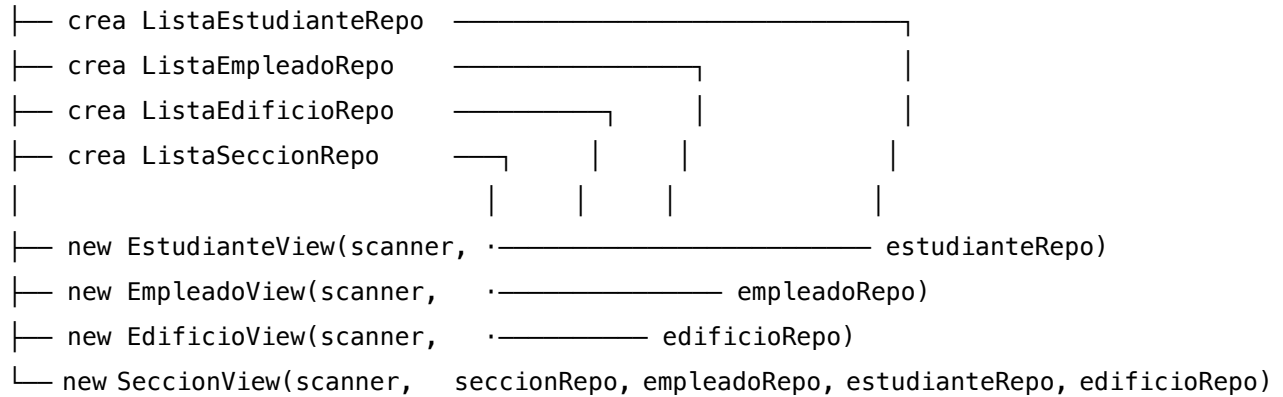
Patrón de repos compartidos

Los repositorios se crean una sola vez en MenuPrincipalView y se pasan a las vistas que los necesitan. Esto garantiza que todas las vistas operan sobre los mismos datos. SeccionView, por ejemplo, recibe los repositorios de empleados, estudiantes y edificios para que su controlador pueda buscar entidades de otros módulos por ID (el docente en empleados, los inscritos en estudiantes y el aula recorriendo los edificios).

Error frecuente: instancias duplicadas. Un mismo repositorio (por ejemplo estudianteRepo) debe ser **la misma instancia** en todas las vistas que lo usan: la opción 1 registra estudiantes y la opción 3 los inscribe en secciones, así que ambas tienen que compartir el objeto creado en MenuPrincipalView. Si una vista hace su propio `new Lista...Repo()` en lugar de recibir el que ya existe, trabajará sobre una lista vacía y aparente: “compila y corre”, pero las secciones nunca encontrarán a los es-

tudiantes ni a los empleados registrados. Reciban los repositorios por constructor; no los creen dentro de la vista.

MenuPrincipalView



SeccionView pasa esos cuatro repositorios a su SeccionController. Como EdificioView y SeccionView comparten la misma instancia de ListaEdificioRepo, las aulas que se creen desde la gestión de edificios quedan visibles al registrar secciones.

2.5 Utilidades y enums

Componente	Descripción
TipoPersonal	Enum existente: CONSERJE, GUARDA, TECNICO, MANTENIMIENTO. Lo reemplaza TipoEmpleado (ver sección 3.1).
Validador	Métodos estáticos: validarEmail, validarFechaIngreso, validarPorcentajeBeca, validarCalificacion.
Conexion	Fábrica de conexiones MySQL. Se activa en P2.
Main	Crea el Scanner y lanza MenuPrincipalView.

2.6 Tests provistos por el docente

Clase de test	Cobertura
EstudianteRegularTest	4 tests: tarifa base, puedeMatricular, getTipo, getInfo.

Clase de test	Cobertura
EstudianteBecadoTest	4 tests: beca 50 %, beca 100 %, beca 0 %, puedeMatricular.

Estos tests deben permanecer en verde en la entrega.

3. Modelo de dominio

Las clases de esta sección no existen en el repositorio. El grupo las diseña e implementa.

3.1 Módulo Empleados (model/personas/)

Empleado: extiende Persona

Un empleado es cualquier persona que trabaja en el centro educativo. Al registrarlo se indica su tipo, lo que determina su rol en el sistema.

Atributo	Tipo	Descripción
salario	double	Salario mensual.
fechaIngreso	LocalDate	Fecha en que comenzó a laborar.
tipo	TipoEmpleado	Categoría del empleado (ver enum abajo).

Método: `getInfo()`: describe al empleado incluyendo su tipo, nombre, apellidos y salario, en formato legible consistente con `EstudianteRegular.getInfo()`.

TipoEmpleado: enum (reemplaza a TipoPersonal)

Valor	Descripción
DOCENTE	Personal docente. Es el único tipo que puede ser asignado a una sección académica.
ADMINISTRATIVO	Personal de oficina y gestión.
GUARDA	Personal de seguridad.
MISCELANEO	Personal de limpieza y servicios generales.
MANTENIMIENTO	Personal de mantenimiento de instalaciones.

3.2 Módulo Académico (model/academico/ y model/infraestructura/)

Identidad: id técnico vs. identificador de negocio. El contrato Repositorio<T> opera con un `int id` (`buscarPorId(int)`, `eliminar(int)`). Por eso Edificio, Aula y Seccion llevan un `id (int)` además de su identificador de negocio (`codigo/numero`, legible para el usuario). El `id` es la clave técnica que usa el repositorio; lo asigna el controller con un contador incremental, igual que `EstudianteController` con `proximoId`. El `codigo/numero` lo escribe el usuario y describe a la entidad; el `id` lo genera el sistema.

Cuidado con el id de las aulas. Como `SeccionController` localiza un aula por su `id` recorriendo *todos* los edificios, ese `id` debe ser **único a nivel de todo el sistema**, no por edificio. Un contador que reinicie en cada edificio (aulas 1, 2, 3... repetidas entre edificios) hará que la búsqueda devuelva el aula equivocada. Mantengan un único contador de aulas compartido (por ejemplo, en el controller que las crea), no uno por Edificio.

Edificio

Espacio físico que agrupa aulas. La relación con `Aula` es de **composición**: las aulas pertenecen al edificio y no existen sin él.

Atributo	Tipo	Descripción
<code>id</code>	<code>int</code>	Clave técnica para el repositorio. La asigna el controller.
<code>codigo</code>	<code>String</code>	Identificador de negocio del edificio o pabellón (ej: "A", "Pabellón B").
<code>nombre</code>	<code>String</code>	Nombre descriptivo (ej: "Edificio Central").
<code>aulas</code>	<code>List<Aula></code>	Aulas que contiene. Empieza vacía.

Métodos: agregar un aula; listar todas las aulas; buscar un aula por número.

Aula

Espacio físico identificado por un número dentro de un edificio.

Atributo	Tipo	Descripción
id	int	Clave técnica única (a nivel de todo el sistema). La asigna el controller al crear el aula. Permite ubicar un aula al registrar una sección.
numero	String	Identificador de negocio del aula dentro del edificio (ej: "101", "Lab-02").
capacidad	int	Cantidad máxima de personas.
tipo	TipoAula	Categoría del espacio (enum, ver abajo).
edificio	Edificio	Edificio al que pertenece. Parte de la composición.

TipoAula: enum

Valores: REGULAR, LABORATORIO, AUDITORIO.

Seccion

Representa un curso en ejecución: tiene un docente asignado, un grupo de estudiantes y está ubicado en un aula.

Atributo	Tipo	Descripción
id	int	Clave técnica para el repositorio. La asigna el controller.
codigo	String	Identificador de negocio de la sección (ej: "PROG3-01").
nombre	String	Nombre del curso (ej: "Programación III").
docente	Empleado	Empleado asignado. Debe ser de tipo DOCENTE.
aula	Aula	Aula donde se imparte.

Atributo	Tipo	Descripción
estudiantes	List<Estudiante>	Estudiantes inscritos. Empieza vacía.

Métodos: agregar un estudiante; remover un estudiante; listar los estudiantes inscritos.

4. Requisitos funcionales

RF-01: Módulo Empleados

RF-01.1 El sistema deberá permitir registrar un empleado solicitando: nombre, apellidos, email, salario, fecha de ingreso y tipo (seleccionado de TipoEmpleado).

RF-01.2 El sistema deberá permitir listar todos los empleados registrados, mostrando su información con `getInfo()`.

RF-01.3 El sistema deberá permitir buscar un empleado por ID.

RF-01.4 El sistema deberá permitir actualizar los datos de un empleado.

RF-01.5 El sistema deberá permitir eliminar un empleado con confirmación previa.

RF-01.6 Ante datos inválidos (campos vacíos, email incorrecto, salario negativo), el sistema deberá mostrar un mensaje de error claro sin terminar el programa.

RF-01.7 La opción 2 del `MenuPrincipalView` deberá quedar conectada al módulo de empleados.

Implementación esperada: `EmpleadoController`, `EmpleadoView` (extiende `VistaBase`), `ListaEmpleadoRepo` (extiende `Repositorio<Empleado>`), siguiendo el patrón del módulo de Estudiantes.

RF-01.8 El grupo deberá escribir al menos 3 tests para `Empleado` que verifiquen la creación correcta y el método `getInfo()`.

RF-02: Módulo Académico

RF-02.A: Edificios **RF-02.A.1** El sistema deberá permitir registrar un edificio solicitando código y nombre.

RF-02.A.2 El sistema deberá permitir listar todos los edificios registrados.

RF-02.A.3 El sistema deberá permitir buscar un edificio por ID y ver las aulas que contiene.

RF-02.A.4 El sistema deberá permitir eliminar un edificio con confirmación previa. **Integridad referencial:** no se podrá eliminar un edificio que todavía tenga aulas; primero deben eliminarse sus aulas. Si el edificio tiene aulas, el controller rechaza el borrado con un mensaje claro.

RF-02.A.5 No se podrá registrar un edificio sin código ni nombre.

RF-02.B: Aulas **RF-02.B.1** El sistema deberá permitir agregar un aula a un edificio existente, solicitando número, capacidad y tipo.

RF-02.B.2 El sistema deberá permitir listar las aulas de un edificio.

RF-02.B.3 El sistema deberá permitir eliminar un aula con confirmación previa.

RF-02.B.4 El aula deberá conocer el edificio al que pertenece (relación de composición).

RF-02.C: Secciones **RF-02.C.1** El sistema deberá permitir registrar una sección solicitando código, nombre, el ID de un aula existente y el ID de un empleado existente. El usuario ingresa los IDs; el controller localiza el empleado en Repositorio<Empleado> y el aula recorriendo los edificios de Repositorio<Edificio> (las aulas viven dentro de su edificio por composición). Si algún ID no existe, rechaza el registro con un mensaje claro.

RF-02.C.2 El sistema deberá rechazar el registro si el ID de empleado no existe o si el empleado encontrado no es de tipo `DOCENTE`, mostrando un mensaje de error claro sin terminar el programa.

RF-02.C.3 El sistema deberá permitir agregar un estudiante a una sección ingresando su ID. El controller valida que el ID exista antes de agregarlo. Si no existe, muestra un error.

RF-02.C.4 El sistema deberá permitir remover un estudiante de una sección ingresando su ID.

RF-02.C.5 El sistema deberá permitir listar las secciones registradas mostrando su aula, docente y cantidad de estudiantes inscritos.

RF-02.C.6 El sistema deberá permitir eliminar una sección con confirmación previa. **Integridad referencial:** no se podrá eliminar una sección que tenga estudiantes inscritos; primero deben removerse los estudiantes (RF-02.C.4). Si la sección tiene inscritos, el controller rechaza el borrado con un mensaje claro.

RF-02.7 La opción 3 del `MenuPrincipalView` deberá quedar conectada al módulo académico.

Implementación esperada:

El módulo académico se divide entre dos integrantes:

- **Edificio + Aula:** EdificioController, EdificioView (extiende VistaBase), ListaEdificioRepo (extiende Repositorio<Edificio>). Mismo patrón que Empleados.
- **Sección:** SeccionController, SeccionView (extiende VistaBase), ListaSeccionRepo (extiende Repositorio<Seccion>).

SeccionController recibe cuatro repositorios por constructor: Repositorio<Seccion>, Repositorio<Empleado>, Repositorio<Estudiante> y Repositorio<Edificio>. Los usa para validar IDs al registrar secciones y al agregar estudiantes. El de edificios es necesario porque las aulas no tienen repositorio propio: viven dentro de su edificio (composición), así que para ubicar un aula por su ID hay que recorrer los edificios. El Controller no le pregunta nada a la vista: solo recibe datos y lanza excepciones si algo es inválido, igual que EstudianteController.

Pista: controller con varios repositorios. A diferencia de EstudianteController (que solo guarda y lee de un repo), aquí el controller **busca entidades en otros repositorios** antes de construir la sección. El esqueleto de la operación es:

```
public SeccionController(
    Repositorio<Seccion> seccionRepo,
    Repositorio<Empleado> empleadoRepo,
    Repositorio<Estudiante> estudianteRepo,
    Repositorio<Edificio> edificioRepo) { ... }

public Seccion registrar(String codigo, String nombre, int aulaId, int docenteId) throws Excepcion {
    // 1. validar codigo/nombre no vacíos
    // 2. buscar el empleado por docenteId en empleadoRepo (Optional)
    //    - si no existe -> throw new IllegalArgumentException(...)
    //    - si su tipo != DOCENTE -> throw new IllegalArgumentException(...)
    // 3. buscar el aula por aulaId recorriendo los edificios de edificioRepo
    //    (cada edificio expone sus aulas) -> si no aparece, lanzar excepción
    // 4. construir la Seccion con el aula y el docente, y seccionRepo.guardar(seccion)
}
```

Para agregar un estudiante (RF-02.C.3) el patrón es el mismo: buscar el ID en estudianteRepo, y si está presente, agregarlo a la sección; si no, lanzar excepción. La vista solo pasa IDs; toda la validación vive en el controller.

RF-02.8 El grupo deberá escribir al menos 3 tests para las clases del módulo académico que verifiquen: la composición Edificio→Aula, el rechazo de un docente con tipo incorrecto, y la búsqueda de un estudiante por ID inexistente.

Decisiones de diseño del grupo: - Cómo organizar la vista del módulo académico (un menú unificado con sub-menús, o vistas separadas por entidad). - Si EdificioView y SeccionView

se lanzan desde `MenuPrincipalView` por separado o desde una vista académica intermedia.

5. Requisitos no funcionales

RNF-01: Formato de código Antes de cada commit ejecutar `mvn fmt:format` sobre los archivos modificados. El tag de entrega debe pasar `mvn fmt:check` sin errores.

RNF-02: Tests `mvn test` debe pasar en verde en el tag de entrega, incluyendo los tests del docente y los del grupo.

RNF-03: Separación de capas El Controller no debe tener referencia a clases de View. La View no debe contener lógica de negocio. Esta separación permite reemplazar el repositorio en memoria por uno MySQL en P2 sin modificar el Controller.

RNF-04: Estructuras de datos Usar `ArrayList` para listas de entidades. No se requieren `HashMap`, `Comparator` ni colecciones avanzadas.

RNF-05: Persistencia de cambios Cada vez que el controller modifique una entidad ya guardada (por ejemplo, al inscribir o remover un estudiante de una sección, o al agregar un aula a un edificio), debe llamar a `repo.actualizar(entidad)`. En P1 el repositorio en memoria guarda referencias, así que el cambio “se ve” aunque no se llame a `actualizar()`; pero en P2 el repositorio MySQL **no** persistirá el cambio sin esa llamada. Tomen el hábito ahora: el controller no debe asumir que mutar el objeto basta.

6. Contexto hacia el Proyecto 2

P1 produce un sistema funcional con almacenamiento en memoria. Los datos desaparecen al cerrar el programa. Es por eso que en el P2 el diseño de las capas en P1 determina qué tan difícil les va a ser el P2. Un Controller que habla directamente con un `ArrayList` hace el cambio inviable.

7. Flujo de trabajo con Git

7.1 Configuración inicial

1. Hacer **fork** del repositorio del profesor en GitHub.
2. Clonar el fork localmente:

```
git clone https://github.com/[su-usuario]/educore.git
cd educore
```

3. Agregar el repositorio del profesor como remote upstream:

```
git remote add upstream https://github.com/jocoto14/educore.git
```

7.2 Durante el desarrollo

Cada integrante trabaja en su propia rama según el módulo asignado:

```
git checkout -b feat/modulo-empleados
git checkout -b feat/modulo-edificio-aula
git checkout -b feat/modulo-seccion
```

Antes de cada commit, formatear los archivos modificados:

```
mvn fmt:format
```

7.3 Entrega

La entrega no se hace mediante Pull Request. Se debe hacer checkout directamente al tag.

1. Mergear todas las ramas a main del fork.
2. Verificar:

```
mvn test
mvn fmt:check
```

3. Crear el tag anotado y pushearlo:

```
git tag -a v1.0-p1 -m "Entrega Proyecto 1 - [Nombres del equipo]"
git push origin v1.0-p1
```

4. Comprobar que `git checkout v1.0-p1` compila y el menú funciona.

El profesor publicó la base como `v0.0`. La entrega del grupo es `v1.0-p1`. El profesor debe ser incluido como colaborador en sus repositorios bajo el username de `jocoto14`.

8. Entregables

Todo debe estar accesible al hacer `git checkout v1.0-p1`.

Repositorio

- Módulos Empleados y Académico implementados según los requisitos de la sección 4.
- `mvn test` en verde.
- `mvn fmt:check` sin errores.
- Historial de commits con trabajo de los tres integrantes desde sus ramas.
- Tag `v1.0-p1` presente y pusheado.

Carpeta /docs en el repositorio

- **README.md** con instrucciones para compilar y ejecutar el proyecto en Java 21. Asimismo como instrucciones de uso.

PDF de documentación

- Portada: nombres de los integrantes, grupo, instrumento, fecha.
- Distribución de tareas: quién estuvo a cargo de qué módulo y cómo coordinaron el trabajo.
- Decisiones de diseño: para cada decisión marcada en la sección 4, qué eligieron y por qué.
- Git: Experiencia de uso de Git y GitHub como herramientas de colaboración.
- Uso de IA: qué herramientas usaron, qué decidieron ustedes, qué aprendieron, qué harían diferente. Si no usaron IA, indicarlo explícitamente. Todo código no generado por ustedes debe ser justificado con: Fuente, por qué fue usado y que ventajas les trajo usarlo.

9. Rúbrica

El proyecto vale el **15 % de la nota del curso**. La rúbrica se expresa sobre **100 puntos**; la nota del proyecto es esos puntos llevados al 15 %.

9.1 Cómo se califica

Cada criterio se divide en ítems concretos. Cada ítem se evalúa con un nivel:

Nivel	Significado
2 — Logrado	Cumple completo y funcional.
1 — Parcial	Cumple lo esencial, con defectos acotados.
0 — No logrado	Ausente, no compila, no corre, o incumple.

La nota de cada criterio es proporcional a los niveles obtenidos. Solo se evalúa lo que esté en el tag v1.0-p1: si mvn test no compila sobre el tag, los ítems de funcionalidad que requieren ejecución no pueden superar nivel 1.

9.2 Pesos por criterio

#	Criterio	Puntos	Trazabilidad
A	Modelo de dominio	25	§3, RF-01, RF-02
B	Funcionalidad MVC	25	RF-01, RF-02, RNF-02
C	Diseño orientado a objetos y separación de capas	15	§3, RNF-03, RNF-05
D	Flujo de Git y entrega	15	§7, RNF-01
E	Documentación en repositorio (README)	10	§8
F	PDF de documentación	10	§8
Total		100	

A · Modelo de dominio — 25 pts

Ítem	Evidencia observable	Origen	Nivel (0-2)
A1	Empleado extiende Persona con salario, fechaIngreso y tipo; getInfo() legible y consistente con el de Estudiante. TipoEmpleado con sus valores reemplaza a TipoPersonal.	§3.1	

Ítem	Evidencia observable	Origen	Nivel (0-2)
A2	Edificio y Aula modelados con composición (el aula vive dentro del edificio); atributos y TipoAula correctos.	§3.2	
A3	Seccion con docente:Empleado, aula:Aula y estudiantes:List, con métodos para inscri- bir/remover/listar estudiantes.	§3.2	
A4	Distinción id técnico (lo asigna el controller) vs. identificador de negocio, con el id de Aula único a nivel de todo el sistema.	§3.2	

Nivel 1: falta un atributo, la composición se modeló como simple asociación, o getInfo() queda inconsistente.

B · Funcionalidad MVC — 25 pts

Verificar ejecutando el programa desde el menú. Cada flujo se prueba con un caso válido y uno inválido.

Ítem	Evidencia observable	Origen	Nivel (0-2)
B1	Módulo Empleados: CRUD completo desde la opción 2, con confirmación previa al eliminar.	RF-01.1–01.5, RF-01.7	
B2	Datos inválidos (campo vacío, email mal formado, salario negativo) muestran error claro sin terminar el programa .	RF-01.6	
B3	Edificios y Aulas: CRUD operativo con integridad referencial (no se borra un edificio que aún tiene aulas).	RF-02.A, RF-02.B	
B4	Secciones: registro validando IDs de aula y empleado, rechazando al empleado que no es DOCENTE ; inscribir/remover/listar estudiantes.	RF-02.C.1–C.5	
B5	Secciones: eliminación con integridad referencial (rechaza si tiene inscritos); opción 3 del menú conectada.	RF-02.C.6, RF-02.7	
B6	mvn test pasa en verde sobre el tag, incluyendo los tests del docente.	RNF-02	

Ítem	Evidencia observable	Origen	Nivel (0-2)
B7	Repositorios compartidos como misma instancia desde MenuPrincipalView (las secciones encuentran empleados, estudiantes y aulas ya registrados).	§2.4	

Nivel 1: el flujo corre con datos válidos pero no valida el caso inválido, no pide confirmación o no respeta la integridad referencial.

C · Diseño orientado a objetos y separación de capas — 15 pts

Ítem	Evidencia observable	Origen	Nivel (0-2)
C1	Herencia correcta (Empleado extends Persona) sin duplicar atributos de la base.	§3.1	
C2	Composición Edificio $\rightarrow$ Aula real: el aula no tiene repositorio propio, vive dentro del edificio.	§3.2	
C3	Controller sin referencia a clases de View: recibe datos y lanza excepciones, no imprime ni lee de consola.	RNF-03	

Ítem	Evidencia observable	Origen	Nivel (0-2)
C4	View sin lógica de negocio: no calcula ni valida reglas de dominio; eso se delega al controller.	RNF-03	
C5	actualizar() se llama tras mutar una entidad ya guardada (inscribir/remover estudiante, agregar aula).	RNF-05	

Nivel 1: separación respetada salvo en una clase (p. ej. un `System.out` en el controller), o `actualizar()` aplicado solo en algunas operaciones.

D · Flujo de Git y entrega — 15 pts

Ítem	Evidencia observable	Origen	Nivel (0-2)
D1	Ramas por módulo en el historial, con commits de los tres integrantes (no un único autor).	§7.2, §8	
D2	Tag anotado <code>v1.0-p1</code> presente, pusheado y con <code>git checkout v1.0-p1</code> compilando.	§7.3, §8	
D3	<code>mvn fmt:check</code> pasa sin errores sobre el tag.	RNF-01	
D4	Mensajes de commit con sentido y trabajo distribuido (no un solo commit masivo).	§7.2	

Ítem	Evidencia observable	Origen	Nivel (0-2)
D5	Profesor agregado como colaborador (jocoto14).	§7.3	

Nivel 1: ramas presentes pero todo el trabajo en una sola, o tag creado pero el checkout no compila.

E · Documentación en repositorio (README) — 10 pts

Ítem	Evidencia observable	Origen	Nivel (0-2)
E1	Instrucciones de compilación y ejecución en Java 21 que funcionan al seguirlas.	§8	
E2	Instrucciones de uso del sistema (cómo navegar el menú y operar los módulos).	§8	

F · PDF de documentación — 10 pts

Ítem	Evidencia observable	Origen	Nivel (0-2)
F1	Portada completa (integrantes, grupo, instrumento, fecha) y distribución de tareas por integrante/módulo.	§8	

Ítem	Evidencia observable	Origen	Nivel (0-2)
F2	Decisiones de diseño marcadas en §4 (organización de la vista académica; lanzamiento de vistas) justificadas con el porqué , no solo descritas.	§4, §8	
F3	Experiencia con Git/GitHub como herramienta de colaboración, con reflexión propia.	§8	
F4	Uso de IA: herramientas usadas, qué decidió el grupo, qué aprendieron y qué harían diferente; todo código externo justificado con fuente y motivo.	§8	

Nivel 1: secciones descriptivas en vez de justificadas (“usamos sub-menús” sin el porqué), o uso de IA mencionado sin análisis.

9.3 Nota final

La nota de cada criterio es proporcional a los niveles obtenidos en sus ítems sobre el máximo posible. La suma de los seis criterios da una nota sobre 100, que equivale al 15 % del curso.

$$\text{Nota P1 (\%)} = (\text{puntos obtenidos} \div 100) \times 15$$